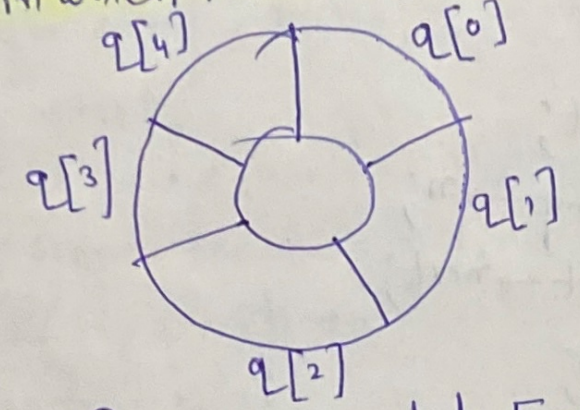
# Circular Queue

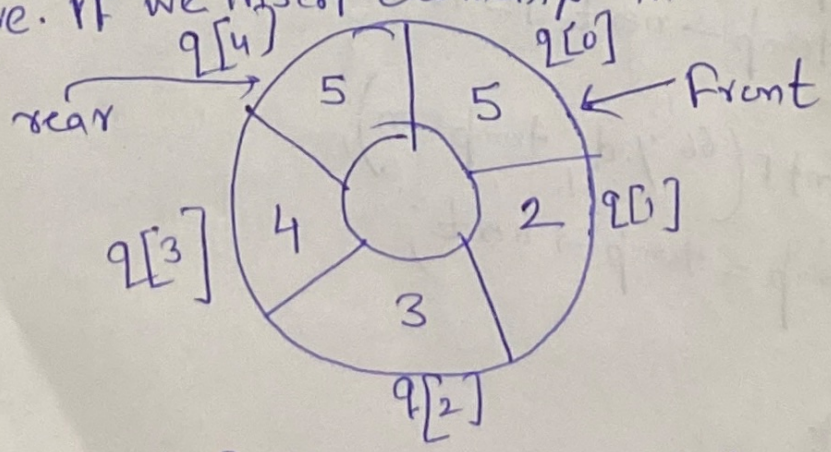
A circular queue is one in which the insertion of an new ele is done at very first location of queue, if last location o of queue is already occupied (in case array representation). In other words, if we have a queue Q of say n elements, then after inserting an element at last location (n-1), the next element will be inserted at first (0). It is possible to insert new elements if and only if those locations are empty. In other words, we can say that in a C-queue the first element comes just after the last element. It can be viewed like a loop of wire in which two ends are connected together.




(C-Queue accomodate 5-elements)

A C-queue overcomes the problem of unutilized space in L-queue implemented as array. In C-queue, front and rear will keep track of elements to be inserted or deleted to maintain the characteristic of an queue.

Let us see the insertion in C-queue: Consider a C-queue as shown above. If we insert elements then rear = rear+1




Now, if queue is full and if we want to add another element as new element is inserted from rear end, the position of element

to be inserted will be calculated by:

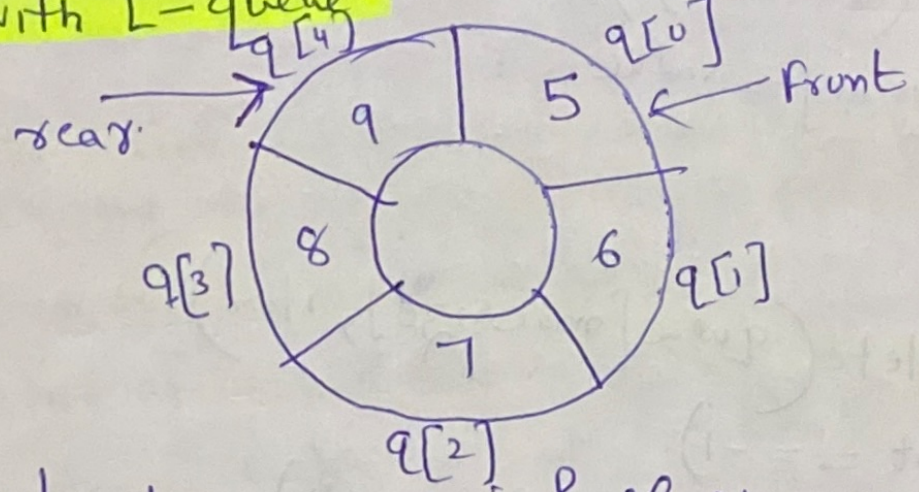
$$rear = (rear + 1) \% maxsize$$

in the current scenario:

$$rear = (4+1) \% 5 = 0$$

In this case, rear points to q[0] where front is already exists. Since, rear and front both are at same location, the overflow condition is satisfied.

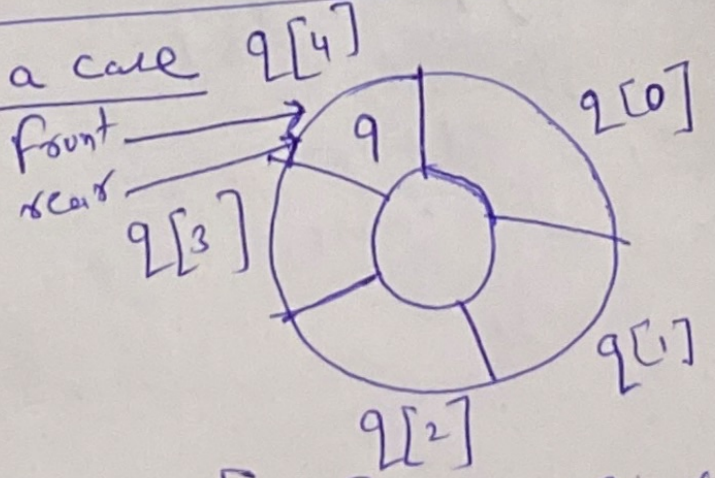
Deletion: The deletion method also requires some modification compared with L-queue.




In this picture, queue is full. Now we want to delete one element front queue. It will be deleted from front end. After deleting, the front end should be modified according to position of front. if front indicate the last element of C-queue then after deleting that element the front should be again reset to 0. It can be done by relation:

$$front = (front + 1) \% maxsize$$

Let us take a case




$$front = [4 + 1] \% maxsize(5)$$

$$front = 0$$

## (Algorithms for insertion & Deletion in C-queue)

C-queue_insert ( queue[maxsize], item)

1. if( Front == (rear+1) % maxsize)
   write (queue over flow and exit)
   else
   if (front == -1)
   set front = rear = 0
   rear = (rear+1) % maxsize
   queue[rear] = value
2. Exit.

C-queue-delete( queue[maxsize], item)

1. if( front == -1)
   write("queue Underflow and exit")
   else
   item = queue[front]
   if(front == rear]
   set front = rear = -1
   else
   front = (front+1) % maxsize

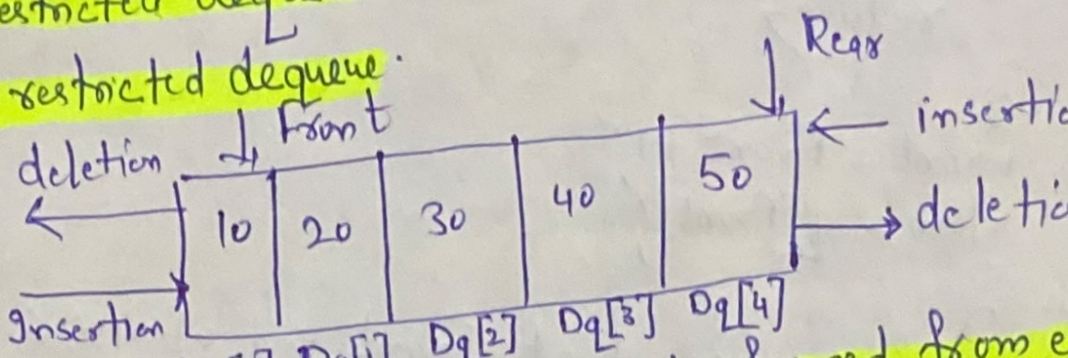
(2) exit.

## Double ended queue (D-queue)

It is a another class of queue in which insertion and deletion operations are performed from both ends. That is, we can insert elements from rear end or from front end both. Hence called D-queue.

### Types of D-queue

1. Input restricted dequeue.
2. Output restricted dequeue.




⇒ Since both insertion and deletion are performed from either end, It is necessary to develop an algo to perform 4-operations!

1. Insertion of an element at rear end
2. Deletion of an element at front end.
3. Insertion of an element at front end
4. Deletion of an element at rear end.

### (1) Algo for insertion of an element at REAR end / Pseudo code

```
void dq_insert_rear( int q[10], int front, int rear, int item,
                     int maxsize)
{
  if (rear == maxsize-1)
  {
    printf("Queue is full");
    return;
  }
  rear = rear + 1;
  q[rear] = item;
}
```

## Deletion of an element at front end

```
void dqdelet_front(int q[10], int front, int rear, int item)
{
if (front == rear)
  {
  printf("Queue is empty");
   return;
  }
front = front+1;
item = q[front];
}
```

## Insertion of an element at front end of queue

```
void dqinsert_front(int q[10], int front, int rear, int item)
{
if(front == 0)
  {
  printf("Queue is full");
  return;
  }
front = front-1;
q[front] = item;
}
```

## Deletion of an element from rear end

```
void dqdelete_rear(int q[10], int front, int rear, int item)
{
if (front == rear)
  {
  printf("Queue is empty");
  return;
  }
rear = rear-1;
item = q[item];
}
```